Lab 6 Submission - OpenMP

Author: Sharique Baig
ERP: 28369

Part 1 — Understanding the OpenMP Execution Model

Task 1: Thread Creation

1. How many threads ran the program? In my run, the program spawned a team of threads. Based on the output, there were 12 threads active (IDs ranging from 0 to 11), which corresponds to the number of logical cores on my machine.

2. Change the thread count using `omp_set_num_threads(4);`. What happens? After adding this line, the output showed exactly 4 threads (Thread 0, 1, 2, and 3) were used, even though my CPU has more.

3. Explain the fork-join model in your own words. The program starts with a single "master" thread. When it reaches a `#pragma omp parallel`, it "forks" into multiple worker threads. Each thread runs its assigned block, and then they all "join" back into the single master thread when the parallel region ends.

Terminal Output for Task 1:

```
Hello from thread 1
Hello from thread 0
Hello from thread 5
Hello from thread 2
Hello from thread 4
Hello from thread 6
Hello from thread 7
Hello from thread 8
Hello from thread 10
Hello from thread 11
```

Part 2 — Parallel Loop Design

Task 2: Work Sharing with `for`

When would dynamic scheduling be better than static? Dynamic scheduling is better when the amount of work in each iteration is different. In static scheduling, a thread might get stuck with a few very heavy tasks while others finish and stay idle. Dynamic lets threads pick up new work as soon as they finish their current task, keeping everyone busy.

Why might scheduling matter in HPC workloads? In HPC, we want to maximize CPU usage. Choosing the right schedule (like dynamic or guided) ensures that no processor is sitting idle, which is key to getting the best performance out of the hardware.

Terminal Output for Task 2 (Dynamic):

```
Thread 1 handled index 0
Thread 1 handled index 12
Thread 1 handled index 13
Thread 1 handled index 14
Thread 1 handled index 15
Thread 1 handled index 16
Thread 1 handled index 17
Thread 1 handled index 18
Thread 1 handled index 19
Thread 5 handled index 5
Thread 7 handled index 9
Thread 8 handled index 6
Thread 2 handled index 1
Thread 6 handled index 11
Thread 3 handled index 2
Thread 9 handled index 10
Thread 0 handled index 4
Thread 4 handled index 3
Thread 10 handled index 7
```

Part 3 — Parallel Tasks vs Parallel Loops

Task 3: Sections

When should you use `sections` instead of `for`? I'd use `sections` when I have a few completely separate, independent tasks to do (like Task A and Task B doing different calculations). A `for` loop is meant for doing the same operation many times across a range of values.

Give a real example of two independent tasks in scientific computing. One task could be updating the physics of a particle simulation while another independent task handles the real-time visualization or logging of the data to a database.

Terminal Output for Task 3:

```
Task A executed by 1
Task B executed by 2
```

Part 4 — Race Conditions and Fixes

Task 4: Broken Counter

Why is `reduction` often faster than `critical`? Because `critical` forces every thread to wait in line to update the counter, causing a bottleneck. `reduction` gives each thread its own private counter that they update locally; the final sum is only calculated once at the very end, which is much more efficient.

When would `atomic` be insufficient? `atomic` only works for single, simple operations like increments. If you have a larger block of code or multiple steps that need to be protected as a single unit, you have to use a `critical` section.

Terminal Output for Task 4 (Broken vs Fixed):

- **Broken:** `Counter value: 3590 (Expected: 10000)`
- **Fixed (Reduction):** `Counter value: 10000 (Expected: 10000)`

Part 5 — Synchronization Concepts

Task 5: Barrier Experiment

What happens if the barrier is removed? The "Before" and "After" prints will mix together. Threads won't wait for everyone to finish the first part, so you'll see some threads start the second print while others are still on the first.

Why are barriers dangerous for performance? They force all threads to stop and wait for the single slowest thread. This wastes a lot of CPU time if there's a big difference in how long threads take to reach the barrier.

Terminal Output for Task 5:

```
Hello before barrier from thread 1
Hello before barrier from thread 2
... (all threads print before)
Hello after barrier from thread 1
Hello after barrier from thread 2
... (all threads print after)
```

Part 6 — Mini Design Challenge / Final Reflection

Task 6: Parallel Prime Counter

1. When does parallelism *not* improve performance? If the task is tiny, the time it takes to create and manage the threads (the overhead) is actually more than the time saved by running in parallel.

2. Which OpenMP construct felt most useful? The `reduction` clause was the most helpful because it handles race conditions automatically for counters, which is a very common task.

3. What design decision matters more than syntax? The design of how the work is distributed and how shared data is handled is much more important than the syntax itself.

Terminal Output for Task 6 (Performance Comparison):

```
--- Serial Version ---
Answer = 17984
Time taken = 0.031000 seconds
```

```
--- Parallel Version ---
Answer = 17984
Time taken = 0.009000 seconds

Speedup: 3.444409 x
```

Final Parallel Code for Task 6:

```c
#include <omp.h>
#include <stdio.h>
#include <math.h>

int unknown_func(int n){
    if(n < 2) return 0;
    for(int i=2; i<=sqrt(n); i++){
        if(n%i==0) return 0;
    }
    return 1;
}

int main(){
    int N = 200000;
    int count = 0;

    #pragma omp parallel for reduction(+:count) schedule(dynamic)
    for(int i=2; i<=N; i++){
        if(unknown_func(i)) {
            count++;
        }
    }

    printf("Answer = %d\n", count);
    return 0;
}
```